

loc vs iloc

在 Pandas 中，`loc` 和 `iloc` 是用于选取 DataFrame 数据的最核心方法。它们的核心区别在于索引的依据不同。

核心区别总结

特性	<code>loc</code> (Label-based)	<code>iloc</code> (Integer position-based)
依据	标签/名称 (行名、列名)	整数位置/下标 (从 0 开始)
切片规则	包含末尾元素 (闭区间)	不包含末尾元素 (左闭右开)
适用场景	知道具体的行名或列名时	知道数据在第几行第几列时
记忆口诀	L ocation (找名字)	Integer L ocation (找数字)

示例演示

首先创建一个带有自定义索引的 DataFrame：

代码块

```
1  import pandas as pd
2
3  df = pd.DataFrame({
4      '姓名': ['张三', '李四', '王五', '赵六'],
5      '年龄': [25, 30, 28, 35],
6      '城市': ['北京', '上海', '广州', '深圳']
7  }, index=['a', 'b', 'c', 'd']) # 设置自定义索引标签
```

```
8
9 print(df)
```

输出：

代码块

	姓名	年龄	城市
1	a	张三	25 北京
2	b	李四	30 上海
3	c	王五	28 广州
4	d	赵六	35 深圳

1. 选取单行/单列

代码块

```
1 # ✓ loc: 使用标签 'a'
2 print(df.loc['a'])          # 选取第一行
3
4 # ✓ iloc: 使用位置 0
5 print(df.iloc[0])          # 选取第一行
6
7 # ✓ loc: 选取“姓名”列
8 print(df.loc[:, '姓名'])
9
10 # ✓ iloc: 选取第 0 列
11 print(df.iloc[:, 0])
```

2. 选取多行/多列（切片）—— 注意边界区别！

代码块 **!** 重点: `loc` 包含末尾, `iloc` 不包含末尾

```
2
3 # loc: 选取标签 'a' 到 'c' 的行 (包含 'c')
4 print(df.loc['a':'c'])
5 # 结果: 返回 a, b, c 三行
6
7 # iloc: 选取位置 0 到 2 的行 (不包含 2, 即只取 0, 1)
8 print(df.iloc[0:2])
9 # 结果: 返回 a, b 两行
```

3. 同时选取行和列

代码块

```
1 # 语法: [行选择, 列选择]
2
3 # loc: 选取标签为 'b' 的行和 '城市' 列
4 print(df.loc['b', '城市']) # 输出: 上海
5
6 # iloc: 选取第 1 行和第 2 列 (下标从0开始)
7 print(df.iloc[1, 2])      # 输出: 上海
```

4. 条件筛选 (只能用 loc)

`iloc` 只能接受整数, 不能直接放布尔条件; 而 `loc` 可以。

代码块

```
1 #  loc: 筛选年龄大于 28 的所有行的“姓名”和“城市”
2 print(df.loc[df['年龄'] > 28, ['姓名', '城市']])
3
4 #  iloc: 这样写会报错
5 # df.iloc[df['年龄'] > 28, ...]
```

💡 避坑指南

1. **默认索引的情况：**如果 DataFrame 没有设置自定义索引（即索引是 0, 1, 2...），那么 `df.loc[0]` 和 `df.iloc[0]` 的结果是一样的。但为了代码清晰，建议根据意图选择。
2. **切片的直觉：**
 - 用 `loc` 时，把它想象成“从 A 班到 C 班”，C 班是包括在内的。
 - 用 `iloc` 时，把它想象成 Python 的列表切片 `list[0:2]`，是不包括最后一位的。
3. **性能：**在处理超大规模数据时，`iloc` 基于位置的查找通常比 `loc` 稍微快一点点，但在日常使用中差异可以忽略不计。

总结一句话：

看到名字（字符串/标签）用 `loc`，看到数字（下标/位置）用 `iloc`。